

Stacks with Python



Follow @techie_programmer for more Programming content!

Introduction

A stack is a linear data structure that follows the **Last-In-First-Out (LIFO)** principle. Think of it like a stack of pancakes - you can only add or remove pancakes from the top.

What is a Stack?

A stack is a data structure that can hold many elements, and the last element added is the first one to be removed. Like a pile of pancakes, the pancakes are both added and removed from the top. So when removing a pancake, it will always be the last pancake you added. This way of organizing elements is called **LIFO: Last In First Out**.

Basic Stack Operations

Basic operations we can do on a stack are:

- **Push:** Adds a new element on the stack
- **Pop:** Removes and returns the top element from the stack
- **Peek:** Returns the top (last) element on the stack
- **isEmpty:** Checks if the stack is empty
- **Size:** Finds the number of elements in the stack

Implementation Methods

Stacks can be implemented by using:

- Arrays (Python lists)
- Linked lists

Stack Uses

Stacks can be used to:

- Implement undo mechanisms
- Revert to previous states
- Create algorithms for depth-first search in graphs
- Enable backtracking

Stacks are often mentioned together with Queues, which is a similar data structure.

Stack Implementation using Python Lists

For Python lists (and arrays), a stack follows this behavior:

```
x = [5, 6, 2, 9, 3, 8, 4, 2]  
Add: → Remove: ←
```

Simple List-Based Stack

Since Python lists have good support for functionality needed to implement stacks, we can create a stack with just a few lines:

```
stack = []
```

```

# Push
stack.append('A')
stack.append('B')
stack.append('C')
print("Stack: ", stack)

# Peek
topElement = stack[-1]
print("Peek: ", topElement)

# Pop
poppedElement = stack.pop()
print("Pop: ", poppedElement)

# Stack after Pop
print("Stack after Pop: ", stack)

# isEmpty
isEmpty = not bool(stack)
print("isEmpty: ", isEmpty)

# Size
print("Size: ", len(stack))

```

Class-Based Stack Implementation

While Python lists can be used as stacks, creating a dedicated Stack class provides better encapsulation and additional functionality:

```

class Stack:
    def __init__(self):
        self.stack = []

    def push(self, element):

```

```

        self.stack.append(element)

    def pop(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack.pop()

    def peek(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.stack[-1]

    def isEmpty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)

# Create a stack
myStack = Stack()

myStack.push('A')
myStack.push('B')
myStack.push('C')

print("Stack: ", myStack.stack)
print("Pop: ", myStack.pop())
print("Stack after Pop: ", myStack.stack)
print("Peek: ", myStack.peek())
print("isEmpty: ", myStack.isEmpty())
print("Size: ", myStack.size())

```

Pros and Cons of List/Array Implementation

Reasons to implement stacks using lists/arrays:

- **Memory Efficient:** Array elements do not hold the next element's address like linked list nodes do
- **Easier to implement and understand:** Using arrays to implement stacks requires less code than using linked lists

Reason for not using arrays:

- **Fixed size:** An array occupies a fixed part of memory. This means it could take up more memory than needed, or if the array fills up, it cannot hold more elements

Stack Implementation using Linked Lists

A linked list consists of nodes with some sort of data, and a pointer to the next node.

A big benefit with using linked lists is that nodes are stored wherever there is free space in memory - the nodes do not have to be stored contiguously right after each other like elements in arrays. Another advantage is that when adding or removing nodes, the rest of the nodes in the list do not have to be shifted.

Linked List Stack Implementation

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Stack:
    def __init__(self):
        self.head = None
        self.size = 0

    def push(self, value):
        new_node = Node(value)
```

```

        if self.head:
            new_node.next = self.head
        self.head = new_node
        self.size += 1

    def pop(self):
        if self.isEmpty():
            return "Stack is empty"
        popped_node = self.head
        self.head = self.head.next
        self.size -= 1
        return popped_node.value

    def peek(self):
        if self.isEmpty():
            return "Stack is empty"
        return self.head.value

    def isEmpty(self):
        return self.size == 0

    def stackSize(self):
        return self.size

    def traverseAndPrint(self):
        currentNode = self.head
        while currentNode:
            print(currentNode.value, end=" -> ")
            currentNode = currentNode.next
        print()

# Usage
myStack = Stack()
myStack.push('A')
myStack.push('B')
myStack.push('C')

```

```
print("LinkedList: ", end="")
myStack.traverseAndPrint()
print("Peek: ", myStack.peek())
print("Pop: ", myStack.pop())
print("LinkedList after Pop: ", end="")
myStack.traverseAndPrint()
print("isEmpty: ", myStack.isEmpty())
print("Size: ", myStack.stackSize())
```

Pros and Cons of Linked List Implementation

Reason for using linked lists:

- **Dynamic size:** The stack can grow and shrink dynamically, unlike with arrays

Reasons for not using linked lists:

- **Extra memory:** Each stack element must contain the address to the next element (the next linked list node)
- **Readability:** The code might be harder to read and write for some because it is longer and more complex

Common Stack Applications

Stacks are used in many real-world scenarios:

- **Undo/Redo operations** in text editors
 - **Browser history** (back/forward navigation)
 - **Function call stack** in programming
 - **Expression evaluation** and parsing
-

Understanding stacks is fundamental to computer science and programming, as they form the basis for many algorithms and are used extensively in software development.